

Lógica de Pronóstico y Optimización

Este notebook contiene las funciones principales para:

- 1. Cargar y preparar los datos.
- 2. Pronosticar la demanda futura.
- 3. Calcular la cantidad óptima de pedido.

0. Importación de Librerías

```
In [1]: import pandas as pd
from statsmodels.tsa.arima.model import ARIMA
import numpy as np
```

1. Carga y Preparación de Datos

```
In [2]: def load_and_prepare_data(data_path='.'):
    """
    Carga, pre-procesa y une los datos de los archivos CSV.

    Args:
        data_path (str): La ruta a la carpeta que contiene los archivos CSV.

    Returns:
        tuple: Una tupla conteniendo los DataFrames para productos, ventas,
              inventario y costos.
    """
    try:
        # Cargar los archivos
        productos = pd.read_csv(f"{data_path}/productos.csv")
        ventas = pd.read_csv(f"{data_path}/ventas.csv")
        inventario = pd.read_csv(f"{data_path}/inventario.csv")
        costos = pd.read_csv(f"{data_path}/Costos_Logisticos.csv")

        # Pre-procesamiento básico
        ventas['fecha'] = pd.to_datetime(ventas['fecha'])

        print("Datos cargados y preparados exitosamente.")
        return productos, ventas, inventario, costos

    except FileNotFoundError as e:
        print(f"Error: No se encontró el archivo {e.filename}. Asegúrate de que los archivos CSV estén en la ruta correcta.")
        return None, None, None, None
```

2. Pronóstico de Demanda (Forecasting)

```
In [3]: def forecast_demand(sales_data, sku, periods=30):
    """
    Pronostica la demanda para un SKU específico.

    Args:
        sales_data (pd.DataFrame): DataFrame de ventas.
        sku (str): El SKU del producto a pronosticar.
        periods (int): Número de períodos (días) a pronosticar.

    Returns:
        pd.Series: Una serie con la demanda pronosticada.
    """
    # Filtrar ventas para el SKU específico y agrupar por día
    sku_sales = sales_data[sales_data['sku'] == sku].copy()
    daily_sales = sku_sales.groupby('fecha')['cantidad_vendida'].sum().resample('D').sum()

    # Rellenar fechas faltantes con 0 ventas
    idx = pd.date_range(daily_sales.index.min(), daily_sales.index.max())
    daily_sales = daily_sales.reindex(idx, fill_value=0)

    if len(daily_sales) < 15:
        # Si hay muy pocos datos, devolver un pronóstico simple (promedio)
        avg_demand = daily_sales.mean()
        return pd.Series([avg_demand] * periods, index=pd.date_range(start=daily_sales.index.max() + pd.Timedelta(days=1), per

    # Entrenar un modelo ARIMA simple
    # El orden (p,d,q) (5,1,0) es un punto de partida común.
    # p: periodos de auto-regresión, d: diferenciación, q: media móvil
    try:
        model = ARIMA(daily_sales, order=(5, 1, 0))
        model_fit = model.fit()
```

```

        forecast = model_fit.forecast(steps=periods)
        # Asegurarse de que el pronóstico no sea negativo
        forecast[forecast < 0] = 0
        return forecast
    except Exception as e:
        print(f"No se pudo entrenar el modelo ARIMA para {sku}: {e}")
        # Fallback a un pronóstico simple si ARIMA falla
        avg_demand = daily_sales.mean()
        return pd.Series([avg_demand] * periods, index=pd.date_range(start=daily_sales.index.max() + pd.Timedelta(days=1), per

```

3. Optimización de Pedido (EOQ)

```

In [4]: def calculate_optimal_order(
        sku,
        forecasted_demand,
        productos_df,
        costos_df,
        scenario='Escenario 1'
    ):
        """
        Calcula la Cantidad Económica de Pedido (EOQ).

        Args:
            sku (str): El SKU del producto.
            forecasted_demand (pd.Series): La demanda pronosticada.
            productos_df (pd.DataFrame): DataFrame de productos.
            costos_df (pd.DataFrame): DataFrame de costos.
            scenario (str): El escenario de costos a utilizar.

        Returns:
            int: La cantidad óptima de pedido, ajustada a las restricciones.
        """
        # 1. Obtener los parámetros del producto y de costos
        product_info = productos_df[productos_df['sku'] == sku].iloc[0]
        cost_info = costos_df[costos_df['Escenario'] == scenario].iloc[0]

        costo_unitario = product_info['costo_unitario']

        # D: Demanda anual
        demanda_anual = forecasted_demand.sum() * (365 / len(forecasted_demand))

        # S: Costo de pedido
        costo_pedido = cost_info['costo_pedido']

        # H: Costo de mantenimiento
        # Se calcula como el % de mantenimiento anual por el costo unitario del producto
        costo_mantenimiento_anual_pct = cost_info['costo_mantenimiento_anual']
        costo_mantenimiento = costo_unitario * costo_mantenimiento_anual_pct

        if costo_mantenimiento == 0:
            return 0 # Evitar división por cero

        # 2. Calcular EOQ (Q)
        # Formula: sqrt( (2 * D * S) / H )
        eoq = np.sqrt((2 * demanda_anual * costo_pedido) / costo_mantenimiento)

        # 3. Ajustar a las restricciones del producto
        cantidad_minima = product_info['cantidad_minima_pedido']
        multiplo = product_info['multiplico_pedido']

        # Aplicar cantidad mínima
        cantidad_ajustada = max(eoq, cantidad_minima)

        # Ajustar al múltiplo de pedido más cercano (redondeo hacia arriba)
        if multiplo > 0:
            cantidad_ajustada = np.ceil(cantidad_ajustada / multiplo) * multiplo

        return int(cantidad_ajustada)

```

4. Ejemplo de Uso y Prueba

```

In [5]: # Ejemplo de uso (para probar las funciones en el notebook)
DATA_PATH = 'C:/Users/Carlos/OneDrive - Universidad Nacional Mayor de San Marcos/Escritorio/PROYECTO DESCRIPTIVO/Casos Ventas

productos, ventas, inventario, costos = load_and_prepare_data(DATA_PATH)

if productos is not None:
    # Probar con un SKU de ejemplo
    sku_ejemplo = 'HM000088'

    print(f"\n--- Ejecutando prueba para el SKU: {sku_ejemplo} ---")

    # 1. Pronosticar demanda
    print("1. Pronosticando demanda para los próximos 30 días...")

```

```

demanda_pronosticada = forecast_demand(ventas, sku_ejemplo, periods=30)
print("Pronóstico:")
print(demanda_pronosticada.head())

# 2. Calcular cantidad óptima de pedido
print("\n2. Calculando cantidad óptima de pedido para 'Escenario 1'...")
cantidad_optima = calculate_optimal_order(
    sku=sku_ejemplo,
    forecasted_demand=demanda_pronosticada,
    productos_df=productos,
    costos_df=costos,
    scenario='Escenario 1'
)
print(f"La cantidad óptima de pedido es: {cantidad_optima} unidades.")

print("\n--- Prueba finalizada ---")

```

Datos cargados y preparados exitosamente.

--- Ejecutando prueba para el SKU: HM000088 ---

1. Pronosticando demanda para los próximos 30 días...

Pronóstico:

2024-01-01 72.076053

2024-01-02 59.541512

2024-01-03 52.519227

2024-01-04 45.181584

2024-01-05 58.183649

Freq: D, Name: predicted_mean, dtype: float64

2. Calculando cantidad óptima de pedido para 'Escenario 1'...

La cantidad óptima de pedido es: 260 unidades.

--- Prueba finalizada ---